Внутреннее файловое хранилище MK61s mini

Версия документа: 11.10.2026

MK61s mini хранит программы, тексты, состояния, шрифты, изображения, ROM виртуальных консолей и каталоги во внешней SPI NOR flash. Постоянным источником истины является собственная файловая система C9 (`program_store`). USB Mass Storage показывает её как динамически сформированный FAT12-диск (`virtual_fat`). Готового FAT-образа во flash нет.

C9 поддерживает только собственный формат. Читателей, миграций и переноса настроек из прежних файловых систем нет. Неподдерживаемая разметка считается неразмеченной: прошивка измеряет flash и создаёт новый том. Файлы и настройки прежнего тома при этом удаляются; `/System` устанавливается заново согласованным с прошивкой комплектом.

Исправная C9 загружается без разрушающего измерения. Повреждённый каталог при валидном C9 locator требует явного `format` (`REPAIR_REQUIRED`). Если валидная C9 содержит другую геометрию той же микросхемы, возвращается `FORMAT_REQUIRED`. Эти состояния защищают новые C9-записи от автоматического стирания.

При форматировании создаются root/WAL и две locator-копии. Пустые страницы inode-таблицы не занимают flash. Старые данные и staging отбрасываются по format epoch и стираются перед повторным использованием.

Эталонная реализация находится в:

- `code/spi_nor_flash.cpp` -- JEDEC/SFDP и низкоуровневый SPI NOR;
- `code/flash_capacity_probe.hpp` -- проверка физической границы;
- `code/storage_geometry.cpp` -- расчёт C9 и FAT12;
- `code/program_store.cpp` -- каталог, WAL, данные, GC и USB staging;
- `code/storage_path.cpp` -- единый разбор абсолютных и относительных путей;
- `code/virtual_fat.cpp` -- FAT12-проекция и импорт изменений хоста.

Основные свойства

Параметр	C9
Физический erase-сектор	4096 байт
Логический сектор USB	512 байт
Поддерживаемая ёмкость SPI NOR	128 КиБ--128 МиБ
Максимальная длина программы/текста	1536 байт; TinyBASIC — 3584 байта
Максимальная длина FMK	1536 байт на F401; 8192 байта на F411
Максимальная длина изображения WBMP	1600 байт
Максимальная длина ROM CHIP-8	3584 байта
Максимальная длина образа MK-61 BIN	4096 байт
Максимальная длина APP-контейнера	20 544 байта
Максимальная длина basename	31 байт M8
Максимальная глубина каталогов	32
Копий FAT	2
Максимум кластеров данных FAT12	4084
Зарезервировано под настройки	1 erase-сектор
Доступно журналу настроек	4080 байт
Одновременно staged USB-блоков	441 по 512 байт на flash от 2 МиБ; до 640 с заимствованием на 512 КиБ

Квота ID независима от FAT-кластеров: `min(physical_sectors * 8, 8192)`. Каждый файл или каталог занимает один ID; продолжения цепочки ID не расходуют. Пустой файл имеет размер и первый FAT-кластер 0, но его имя занимает место.

Физическая flash	Узлов C9	FAT-кластеров	Размер кластера	Секторов FAT12
512 КиБ	1024	4039	512 байт	4096
16 МиБ, W25Q128	8192	4084	4 КиБ	32729

Flash 512 КиБ–1 МиБ получает виртуальный USB-том 2 МиБ: место для AppleDouble и других временных цепочек хоста. macOS резервирует около 4 КиБ на каждый AppleDouble рядом с обычным файлом. Эти цепочки остаются в live FAT до выхода, а содержимое sidecar не записывается в C9. Виртуальный объём не является обещанием вместить 2 МиБ payload: physical COW allocator продолжает проверять настоящую NOR. На mini 16 МиБ виртуальная геометрия не меняется.

Число ID — предел метаданных, а не обещание вместить столько файлов любого размера. Реальная запись дополнительно ограничена root/LFN entries, FAT-кластерами, физическими записями, COW/WAL, staging и запасом GC. Каталог занимает один ID и столько FAT-кластеров, сколько нужно его entries.

Лимит программ и текстов равен 1536 байтам, кроме исходников TinyBASIC: они могут занимать 3584 байта и всё ещё помещаются с метаданными в один 4-КиБ сектор C9. F401 сохраняет предел 1536 байт для FMK. На F411 шрифт может занимать до 8192 байт. При выборе `SETUP.APP` временно читает исходный FMK, проверяет и разворачивает его в индексированный PFK2; в `BULK` остаётся только готовый для рендеринга образ. Он должен уместиться в 1536 байт на F401 или 8192 байта на F411. Если FMK больше 1600 байт, C9 хранит его тем же потоковым многосекторным способом, что крупные APP и ROM. Только изображения WBMP получают 1600 байт: дополнительные 64 байта позволяют хранить полный кадр 192x64 вместе с заголовком. При 512-байтном FAT-кластере такой файл требует четыре кластера одной цепочки. Сырой ROM CHIP-8 может занимать до 3584 байт - остаток 4-КиБ адресного пространства после точки загрузки `0x200`. APP выделен в отдельный потоковый тип: 64-байтовый заголовок плюс до 20 КиБ сжатого или несжатого образа. Фиксированного числа APP или ROM нет; предел задают общие узлы, блоки и каталоги.

Определение физической ёмкости

Быстрый путь при обычной загрузке

Драйвер читает JEDEC ID командой `0x9F`, затем заголовок и Basic Flash Parameter Table SFDP командой `0x5A`. Значения сохраняются как заявленная геометрия и помогают выбрать протокол команд, но не ограничивают поиск: они не считаются доказательством физической ёмкости и могут быть ошибочными в обе стороны.

Если обе locator-копии уже относятся к текущему каталогу C9 (версия 9), загрузка является read-only:

1. Проверяются сигнатура, версия, committed-state и CRC32 locator.
2. Геометрия полностью пересчитывается и сравнивается с сохранёнными полями.
3. Сравниваются JEDEC ID и заявленная JEDEC/SFDP-граница.
4. На измеренном дальнем конце читается guard сектора настроек.
5. Драйвер переключается на сохранённую измеренную ёмкость.

Ни erase, ни тестовая запись на штатной загрузке не выполняются. Это сохраняет ресурс flash. Завышенная маркировка, однажды измеренная как меньшая физическая ёмкость, также не вызывает повторное форматирование при каждом включении. Единственная штатная запись на этом пути — доведение до двух согласованных locator-копий уже валидного C9; файлы при этом не переписываются. Исправная копия при восстановлении второй не стирается, включая повторный обрыв питания.

Деструктивный пробник неразмеченной flash

Пробник запускается, когда нет ни валидного C9 locator, ни текущего C9 с изменённой геометрией. Содержимое такой микросхемы всё равно будет заменено новой C9, поэтому тестовые секторы не копируются и не восстанавливаются.

Стандартные ёмкости SPI NOR являются степенями двойки. Независимо от заявленной ёмкости по показателям степени от 17 до 27 выполняется настоящий двоичный поиск максимальной рабочей границы:

```
128 КиБ, 256 КиБ, 512 КиБ, ... 64 МиБ, 128 МиБ
```

Для проверки candidate используются два сектора:

```
lower = candidate / 2 - 4096
upper = candidate     - 4096
```

Оба сектора стираются, затем в них записываются разные комплементарные 32-байтные метки. После обеих записей обе метки читаются и сравниваются. Если старшие адресные биты отсутствуют и `upper` зеркалит `lower`, вторая запись потребует невозможного для NOR перехода 0 -> 1 либо финальное сравнение увидит alias. Ошибка команды, verify или таймаут также означает нерабочую границу.

Для диапазона 128 КиБ--128 МиБ требуется не более четырёх проверок candidate, то есть не более восьми основных erase-операций. При проверке границ выше 16 МиБ дополнительно защищаются два низких shadow-сектора. Это обнаруживает трёхбайтную микросхему, которая проигнорировала `EN4B` и приняла четвёртый байт адреса за данные. Таких защитных стираний не более шести за весь поиск; общий худший случай -- 14, а для микросхем до 16 МиБ их нет.

Стирания и page-program одной SPI NOR нельзя выполнять параллельно: кристалл выставляет `BUSY`. Реализация не делает лишнего промежуточного чтения -- обе метки программируются последовательно, а затем проверяются одним этапом чтения.

Поиск всегда включает 128 МиБ, поэтому заниженные JEDEC/SFDP не оставляют неиспользованную физическую ёмкость. Если не проходит даже 128 КиБ, C9 не создаётся.

Адресация больше 16 МиБ

Для микросхем с четырёхбайтным адресом драйвер разбирает optional SFDP 4-Byte Address Instruction Table (`4BAIT`). Если таблица одновременно объявляет обычные read, page-program и выбранный 4-КиБ erase type, используются stateless-команды `13h`, `12h` и указанный таблицей erase opcode. В остальных случаях драйвер разбирает BFPT DWORD 16 и использует `B7h`/`E9h`, в том числе вариант с обязательным `WREN`. Неизвестный способ переключения не угадывается: граница выше 16 МиБ в таком случае не принимается.

SFDP также задаёт размер program-page. Драйвер никогда не отправляет за одну операцию больше 256 байт и уменьшает порцию для микросхем с меньшей страницей, исключая page-wrap.

Горячий путь SPI использует буферные transfer-операции STM32 HAL: сектор читается одним вызовом, page-program -- одним вызовом на физическую страницу, а verify 512 байт -- одной READ-командой и одной порцией сравнения. Побайтового входа в HAL для каждого из 512 байт нет. Постоянная RAM при этом не расходуется; для verify используется ограниченный 512-байтный stack buffer.

Что именно гарантирует быстрый тест

Разреженный пробник доказывает адресуемый диапазон, отсутствие зеркалирования на проверяемых границах и работоспособность последнего сектора, отданного настройкам. Он намеренно не является полным тестом здоровья каждого сектора: такой тест потребовал бы стереть и записать всю микросхему и занял бы минуты.

Все реальные записи C9, включая locator, каталоги, WAL, данные и настройки, проверяются чтением. Поэтому отдельный изношенный сектор приводит к явному отказу операции, а не к подтверждению повреждённых данных; неиспользованные data-секторы проверяются лениво при первом обращении.

Единицы `getCapacity()`

`getCapacity()` возвращает байты. Старый диагностический код ошибочно печатал после числа суффикс `K`:

```
16777216  = 16 МиБ, W25Q128, а не 16777216 КиБ
524288    = 512 КиБ, а не 524288 КиБ
```

Новый вывод явно пишет `bytes` и отдельно показывает заявленную верхнюю и измеренную C9-ёмкость.

Динамическая карта SPI NOR

Все числа ниже -- номера 4096-байтных erase-секторов. Ни один сектор после измеренной границы не используется.

```
0                       locator A
1                       locator B
2 .. scratch_first-1    metadata-only reserve
scratch_first ..        import scratch (excluded from allocators)
data_first ..           packed data log + movable catalog
... settings-stage      USB staging log
capacity/4096 - 1        settings journal + C9SG guard
```

Два metadata-only резерва вмещают одновременно живой и незавершённый checkpoint: каждый состоит из `ceil(max_nodes * 26 / 4096)` страниц таблицы, одного header-сектора и двух WAL-секторов. Root/WAL/страницы могут вращаться как в этом резерве, так и в свободной части data log. Поля `catalog_a_sector`, `catalog_b_sector`, `catalog_bank_sectors` сохранены в `Geometry` ради APP ABI: это границы резерва, а не текущий адрес каталога.

Между резервом каталога и data log находится отдельный import scratch. На F411 ID plan/name claims логически пусты при начале import; страницы стираются при первой записи. Неинициализированная страница читается как `0xFF`, даже если физически содержит старый plan; запись через границу инициализирует обе страницы. F401 сохраняет компактный eager erase путь, чтобы оставить обязательный ROM reserve.

Состав import scratch: CRC-окна обратной карты `{owner, index, next}`, hash-index исходных имён, план target/source ID, очередь пустых файлов и проверка уникальности имён. Ни data allocator, ни каталог, ни staging эту область не используют. Scratch восстанавливается из committed C9 и принятого host-пакета; источником истины остаются inode и неизменяемые цепочки.

Staging получает 65 секторов на flash от 2 МиБ, 17 на 512 КиБ и 4--8 на меньших томах. Два data-сектора удерживаются для append/GC progress.

Поле	512 КиБ	16 МиБ
Физических секторов	128	4096
Страниц таблицы	7	52
Секторов metadata-only reserve	20	110
Секторов import scratch	17	48
Первый сектор данных	39	160
Секторов данных	71	3870
Первый сектор staging	110	4030
Секторов staging	17	65
Сектор настроек	127	4095

На 512 КиБ после двух секторов GC reserve остаётся 276 КиБ data log. В эти байты входят имена, mapping, headers и transient COW-записи; это не объём гарантированного пользовательского payload.

Locator и смена микросхемы

Две независимые locator-копии `C9FS` находятся в секторах 0 и 1. Каждая хранит:

- версию формата и committed-state;
- измеренную ёмкость и format epoch;
- все рассчитанные границы областей;
- число узлов и FAT-геометрию;
- заявленную JEDEC/SFDP-границу и JEDEC ID;
- CRC32 всей записи.

На дальнем конце находится независимый `C9SG` guard с измеренной ёмкостью и CRC32. Он обнаруживает уменьшение flash, перенос образа на другую ёмкость и простое адресное зеркалирование.

Изменение JEDEC ID, заявленной JEDEC/SFDP-границы, измеренной ёмкости или C9 capacity guard означает замену/несоответствие микросхемы. Тогда граница измеряется заново и создаётся новая C9. Настройки при этом стираются, потому что их физический сектор мог переместиться.

Если отличается только рассчитанная новой прошивкой C9/FAT-геометрия, но совпадают chip ID, заявленная и измеренная ёмкость, дальний settings-sector и его CRC guard, деструктивный пробник не запускается: прошивка ждёт явного форматирования, сохраняющего 4080 байт настроек. Повреждение каталога при валидном C9 locator, напротив, никогда не запускает форматирование. C9 переходит в `REPAIR_REQUIRED`, оставляет flash неизменной и ждёт явного подтверждённого форматирования; settings-sector при этом остаётся доступен.

Каталог C9 и FAT-цепочки

ID стабилен и имеет ширину 16 бит. Один inode занимает 26 байт во flash и 28 байт в RAM. Помимо адреса данных, длин, parent/sibling ссылок, hash имени, kind/type, flags и `exported_size`, он хранит 32-битную ссылку на mapping. Для файла `first_child` не используется; для каталога это голова списка детей.

`data_len` — логические байты API (M8 для текста), `exported_size` — байты FAT (UTF-8 для текста), `record_len` — физическая длина малой записи. Внешний размер считается при записи и публикуется вместе с данными. Rename, GC и checkpoint сохраняют его без декодирования текста. Флаги large/ZX0 имеют значения 1/2; служебный флаг 4 означает освобождение старого FAT-назначения во время import, при этом прежние данные остаются достижимыми до публикации замены.

Таблица состоит максимум из 52 страниц по 4 КиБ. Checkpoint переписывает только затронутые страницы; inode на границе обновляет обе. Передача идёт кусками по 512 байт. Manifest `C9CT` занимает 512 байт,

хранит поколение, эпоху, metadata, адреса/CRC страниц, WAL и allocator cursor. Последним программируется ACTIVE; старое поколение и pending-страницы до этого закреплены.

WAL W9 публикует до 16 inode за одну 512-байтную запись. Между checkpoint помещается 15 транзакций. RAM содержит table cache (512 байт на F401, два блока по 512 байт на F411) и 32 overlay entries. Locator/settings при checkpoint не переписываются. При загрузке проверяются самое новое committed-поколение, все его страницы и WAL. Повреждение последнего root не маскируется выбором более старого.

Неизменяемая F9CH v1 содержит 16-байтный header и список 16-битных FAT-кластеров. CRC защищает header и список; проверяются owner ID, count, диапазон и отсутствие повторов. Запись не пересекает 4-КиБ сектор, её физический предел — 2032 элемента. Максимальный APP на 512-байтном FAT-кластере занимает 41 элемент одной цепочки. Обновление сначала записывает данные и mapping, затем публикует две ссылки одним WAL. Pending pins защищают обе записи от GC. Unlink одного inode атомарно делает старые записи мусором, без длинной транзакции по продолжениям.

Обратная FAT-карта во flash строится окнами по 32 кластера и имеет CRC каждого окна. fat_cluster_info получает владельца, позицию и следующий кластер, не обходя все inode для каждого FAT entry. Отдельный hash-index исходных имён позволяет связать host-объект с прежним ID; совпадение hash подтверждается именем. Последовательный native просмотр каталога использует sibling iterator; поиск нативного имени остаётся линейным по непосредственным детям.

Кеши чтения и распределения

На F411 маленькие файлы могут использовать свободную память OVERLAY. Во время USB закреплённый staging занимает OVERLAY, поэтому проверенный файл может занять свободную APP-арену. Загруженный или загружаемый APP не вытесняется ради чтения; новый APP может вытеснить простаивающий кеш. MK61_PROGRAM_STORE_READ_CACHE_BYTES по умолчанию равен 4096; на F401 кеш выключен. Lease занимает размер файла и 8 байт экспортного курсора. Перед публикацией кеша проверяются CRC, декодирование ZX0 и логическая длина всего payload. Повторное чтение диапазона копирует проверенный payload из RAM; поиск inode и проверка его текущих метаданных выполняются отдельно.

Ключ содержит format epoch, ID, физический адрес, длины, родителя, hash имени, kind/type и flags. Любая попытка физической записи или стирания, включая неудачную, сбрасывает пригодность кеша. Init и format освобождают lease. Streaming, большие файлы, EOF и нехватка RAM используют обычный проверяемый reader.

Этот кеш имеет низкий приоритет. Он не вытесняет загруженный APP ради чтения. APP-аллокатор может освободить простаивающий кеш и один раз повторить выделение; заполнение кеша и живой USB staging защищены.

MK61_C9_ALLOCATION_CACHE на F411 добавляет карту занятых FAT-кластеров (511 байт) и карту живых физических секторов (512 байт). Первая строится после полной проверки CRC цепочек и уникальности владельцев. Изменения учитываются только после durable WAL publication; ошибка доказательства делает карту непригодной. Удаление и освобождение цепочки сбрасывают её для следующей проверки. При исключении старой цепочки перед заменой её CRC проверяется заново, прежде чем убрать биты занятости.

Физическая карта используется для flash до 16 МиБ. Для большей ёмкости остаётся оконный обход inode. Pending records, текущий сектор, reserve, каталог и заимствованные staging-секторы проверяются дополнительно; карта не разрешает стирать эти записи. На F401 обе постоянные карты выключены, а тот же безопасный allocator работает без кешей.

На F411 WORKSPACE, включённый BULK, VM image cache и MSC BOT transport получают стабильную память при первом foreground использовании. Размеры этих буферов сохранены. Адрес и данные остаются до reset; после release WORKSPACE не обнуляется. Выделение heap запрещено в IRQ и во время передачи владения lease. MSC handle готовится до USB enumeration, поэтому SET_CONFIGURATION не требует malloc из прерывания. F401 сохраняет статические буферы.

ELF APP gate вычитает из общей динамической области все будущие retained выделения вместе с запасом allocator overhead. Он проверяет совместное размещение максимального APP и полного staging index после этих резервов, а не только размер свободной памяти сразу после загрузки.

Физический журнал данных, RAW/ZX0 и отсутствие хвостов по 4 КиБ

Сектор данных имеет 16-байтный header `C9D0`. Далее без кластерного выравнивания упаковываются записи:

```
16-byte record header + M8 basename + RAW или ZX0 payload
```

Запись не пересекает erase-сектор, но файл не получает отдельный 4-КиБ сектор. В header хранится физический `stored_len`; CRC покрывает header, имя и именно сохранённые байты. Для RAW `stored_len == data_len`, для ZX0 он меньше. Максимальный малый payload равен 1600 байтам, поэтому запись с максимальным именем занимает `16 + 31 + 1600 = 1647` байт, поэтому даже две максимальные записи гарантированно помещаются в один сектор. Маленькие записи упаковываются плотнее. Неиспользуемым остаётся только хвост, в который уже не помещается следующая конкретная запись; фиксированной потери 4 КиБ на файл нет.

FOCAL, TinyBasic, TEXT, state, Markdown, WBMP и CHIP-8 пробуются как единый поток ZX0 с окном 256. Сжатие выбирается, только если одновременно экономит не менее 64 байт и 10%. MK61 остаётся RAW из-за горячего 64-байтного произвольного чтения; FMK2 остаётся RAW, потому что уже сжат поглифово; APP использует только собственное сжатие внутри контейнера. Бинарный образ MK-61 (`M3`, `.bin`) тоже остаётся RAW на уровне C9: он уже содержит ZX0.

Выбор small/large выполняется после сжатия по `stored_len`. Поэтому логически большой CHIP-8 может стать обычной малой записью. APP, CHIP-8 и MK61_BINARY с физическим payload больше 1600 байт хранятся иначе. Обычная запись такого файла содержит небольшой дескриптор `C9L0` v2: логическую и физическую длину, поколение, CRC всего физического потока и номера секторов. Прежние версии дескриптора не читаются. Данные лежат в динамически выбранных 4-КиБ блоках `C9B0`; 32-байтовый header каждого блока защищает владельца, номер блока, длину, поколение и CRC payload. Это не отдельный раздел и не набор фиксированных слотов: блоки берутся из общего журнала данных и возвращаются сборщику мусора после удаления или замены большого файла.

Новая версия большого файла сначала потоково записывается в свободные `C9B0`. ZX0 encoder пишет туда физический поток напрямую, не создавая второй максимальный буфер. Затем дописывается дескриптор, а ссылки inode на данные и F9CH публикуются одной WAL-транзакцией. До commit старая версия остаётся единственной достижимой; после commit новые блоки становятся живыми, а старые -- мусором. Незавершённые блоки не считаются файлами и могут быть стёрты повторной записью.

Чтение ZX0 начинает декодирование с логического нуля даже для `read_range()`, потому что match зависит от предшествующей истории. Декодер хранит 256-байтное кольцевое окно, выдаёт только требуемый диапазон, но продолжает до конца и одновременно проверяет CRC физических байтов, точную логическую длину, end marker и отсутствие хвоста. Для large storage тот же reader переходит между `C9B0` и проверяет CRC каждого блока. Последовательный входной поток читается временными блоками по 512 байт: блок находится на стеке только во время операции, поэтому ускорение не добавляет постоянную RAM.

Обновление и rename дописывают новую запись. Rename переносит физический payload без распаковки/повторного сжатия и пересчитывает record CRC. После WAL commit inode начинает указывать на новую запись; старая версия становится мусором. Повторная запись полностью идентичных логических данных является no-op и не расходует ресурс flash.

GC использует отдельный reserve-sector. Поиск свободного сектора на F411 может использовать проверенную физическую карту. На F401 и для большой flash сохраняется оконный обход. Выбор victim по объёму живых данных идёт окнами по 32 сектора: таблица inode читается один раз на окно, а не заново для каждого сектора. Живые записи victim копируются в reserve и их новые адреса публикуются атомарными WAL-пакетами до 16 inode. Список переносимых inode не занимает `shared_scratch`: GC продолжает последовательный обход каталога после каждого пакета. Поэтому он может запускаться из rename, пока там лежит сохранённый физический payload. Только после публикации всех адресов victim стирается и становится новым резервом. Потеря питания оставляет старые либо уже committed новые ссылки, но не промежуточный inode.

Host-тесты принудительно обрывают каждую отдельную program/erase-операцию на границах checkpoint, настоящего GC почти полного журнала и компактизации USB staging. После каждого возможного обрыва выполняются reboot, полная проверка старой либо новой committed версии и повтор операции. Для успешного пути также проверяются верхние пределы числа мутаций и erase, чтобы корректность не

скрывала неожиданное ухудшение времени работы или износа.

Для ZX0 дополнительно выполняется fixed-seed mixed-order churn с RAW, small ZX0 и large ZX0: записи, замены, rename и delete перемешаны, каждые 13 операций моделируется reboot и всё хранилище сверяется с RAM-моделью. Замена логически многокластерного, но физически small CHIP-8 отдельно проверяется при полностью занятой квоте inode: цепочка переиспользуется без временного свободного ID. Отдельная матрица перебирает обрыв после каждой flash mutation при замене small и large ZX0 и требует целиком старую либо целиком новую версию. Дополнительный граничный тест заполняет минимальный том 30 файлами и заставляет GC выбрать и стереть сектор исходной ZX0-записи непосредственно внутри rename; после reboot проверяются имя, физический размер, флаг и логические данные.

Каталоги и имена

C9 поддерживает создаваемые пользователем каталоги с произвольной структурой. Каталог хранит head двусвязного списка непосредственных детей, поэтому его просмотр не требует читать или сортировать всё оглавление flash. Максимальная глубина -- 32. Проверка учитывает всё поддерево и не позволяет обойти лимит переносом глубокого каталога.

Внутреннее имя -- валидный M8 basename длиной 1--31 байт. Каждый байт равен одному символу; UTF-8 существует только на границе USB/MKC. Запрещены:

- управляющие байты и FAT-символы `< > : " / \\ | ? *`;
- `.` и `..`;
- завершающие пробел или точка;
- DOS device names `CON`, `PRN`, `AUX`, `NUL`, `COM1`--`COM9`, `LPT1`--`LPT9`, в том числе перед расширением.

В пределах каталога не допускаются имена, которые совпадают без учёта регистра. Сравнение складывает регистр непосредственно в M8: ASCII, кириллицу и символы CP1251, для которых есть пара регистров. Например, Каталог конфликтует с каталог, каталог `demo.m61` -- с MK61-файлом basename `demo`, а `Report.txt` -- с `report.txt`.

Файлы получают расширение, публичную двухбайтовую сигнатуру и отдельную квоту по типу:

Тип C9	Magic	Расширение USB	Максимум	Назначение
MK61	`M1`	`.m61`	1536 байт	сценарий МК-61
MK61_BINARY	`M3`	`.bin`	4096 байт	CRC32 + ZX0, загрузка в банки МК-61
FOCAL	`F1`	`.foc`	1536 байт	исходник FOCAL
Tiny BASIC	`B2`	`.tbi`	3584 байта	исходник Tiny BASIC
TEXT	`T1`	`.txt`	1536 байт	обычный текст
Markdown	`T2`	`.md`	1536 байт	M8 Markdown
MK61 state	`M2`	`.state.txt`	1536 байт	состояние калькулятора
FONT	`f1`	`.fmk`	1536 байт F401; 8192 байта F411	графический шрифт FMK2/M8
IMAGE1	`I1`	`.wbmp`	1600 байт	стандартный WBMP Type 0
APP	`A1`	`.app`	20 544 байта	исполняемый контейнер F401
CHIP8	`C1`	`.ch8`	3584 байта	сырой ROM CHIP-8

Magic является свойством уже известного типа C9. Он нужен для отображения типа и поиска зарегистрированного APP-обработчика, но не заменяет тип inode. Первый ASCII-байт задаёт семейство: `C`

означает console. Вторая консольная платформа получит C2, а не ещё одно частное расширение в Проводнике. Регистр значим, поэтому F1 и f1 различаются.

IMAGE1 — только внутреннее имя типа C9. Содержимое записи не получает собственного заголовка: это неизменённый стандартный файл WBMP, который можно копировать с диска и открывать обычными программами. В short entry FAT 8.3 четырёхбуквенное .wbmp представлено как .WBM; этот вариант принимается и при обратном импорте без LFN. Формат файла, создание через ImageMagick, управление просмотрщиком и флаг его условной компиляции описаны в MK61s-mini-WBMP.pdf.

MK61_BINARY передаётся через USB и fsput побайтно, без M8/UTF-8 и повторного сжатия. Он загружается явной командой load <адрес> <путь.bin>; самостоятельное открытие .bin в Проводнике не запускает его. Формат описан в руководстве терминала.

CHIP8 также не добавляет заголовок к содержимому. В .ch8 лежит стандартный ROM, который загружается по адресу 0x200; байты C1 находятся только в таблице типа и регистрации обработчика. Формат, экран, управление и критерии совместимости описаны в MK61s-mini-CHIP8.md.

APP имеет единое расширение и общий 64-байтовый заголовок MK61APP. Поле kind различает пользовательское приложение и системные сервисы FOCAL, TinyBASIC, WBMP viewer, Markdown viewer, CHIP-8, SETUP и USBDISK; отдельного расширения для системных компонентов нет. Канонические имена /System/FOCAL.APP, /System/BASIC.APP, /System/WBMP.APP, /System/MARKDOWN.APP, /System/CHIP8.APP ищутся в обычном видимом каталоге C9, когда соответствующий компонент выбран как APP. Каждый из пяти компонентов можно независимо встроить, вынести в APP или выключить; штатный F411 их встраивает, штатный F401 выносит в APP. Пользовательские APP можно хранить в любом другом каталоге. На F401 к этому списку добавляются /System/SETUP.APP и /System/USBDISK.APP. На F411 SETUP и USB-диск встроены по умолчанию, но выносятся в соответствующие файлы /System ключами MK61_SETUP_AS_APP=1 и MK61_EXTERNALIZE_USBDISK=1. ABI 6 не привязан к CRC конкретной прошивки, использует таблицы C API и арендует размер своего образа/BSS, округлённый до 32 байт, сверху свободной RAM, ниже защиты стека. 20 КиБ ограничивают размер одной APP; постоянного резерва нет. После выгрузки блок возвращается в свободный участок. 0x20000000 — только виртуальная база линковки; таблица релокаций исправляет внутренние адреса при загрузке в выбранный блок. Payload сжат ZX0, при выигрыше размера применяется Thumb BCJ. После потока находится компактная таблица релокаций под общей CRC. HELP0.TXT и HELP1.TXT в /System — обычные текстовые файлы справки с подписью набора команд; их генерируют вместе с прошивкой. В комплекте для копирования через USB-диск они записаны в UTF-8; при извлечении тома C9 переводит текст в свой внутренний M8. Терминальная загрузка fsput обходит USB-перекодировку и передаёт эти же страницы сразу в M8. ABI 2/3/4 больше не исполняются. Исходный код и сборка контейнера описаны в MK61s-mini-APP-Programming.md.

Поле handled_type_magic заголовка APP может регистрировать обработчик типа. Штатный CHIP8.APP регистрирует C1. Отдельный WBMP.APP регистрирует I1, когда Markdown выключен; графический Markdown сам обслуживает I1 и на WS0010 100x16. Графический MARKDOWN.APP регистрирует T2, а resident направляет в него также I1. Это не content magic контейнера или открываемого файла.

Проводник, редакторы и общий файловый диалог

Проводник работает с непосредственными детьми текущего каталога и не строит в RAM плоский список всего диска. Короткое OK входит в папку, загружает M61, запускает файл соответствующего языка либо пользовательский APP. Для остальных запускаемых типов он берёт двухбайтовый magic из таблицы типа и вызывает встроенный или APP-обработчик: I1 открывает WBMP, T2 — Markdown, C1 запускает CHIP-8.

Выбранная строка помечена постоянным > слева, как в остальных меню. Файл показан как имя.ext, каталог — как имя/: суффикс идёт сразу после имени, а не в отдельной колонке справа. Длинное имя выбранного пункта прокручивается вместе с суффиксом. Такое же оформление используется в общих диалогах выбора файла и каталога, во встроенном и APP-проводнике.

Если в папке непосредственно лежит autoexec.m61 или autoexec.tbi, Проводник запускает этот M61- или TinyBASIC-сценарий сразу после входа коротким OK и выходит к основному экрану. При наличии обоих файлов приоритет имеет autoexec.m61; имя сравнивается без учёта регистра. Автозапуск не выполняется при открытии корня Проводника, возврате в родительскую папку через ESC и навигации внутри общих диалогов выбора файла или каталога. После выхода и повторного входа в папку сценарий запускается снова.

Долгое OK открывает действия:

- просмотр или запуск/загрузка файла;
- редактирование FOCAL и Tiny BASIC;
- создание произвольной папки;
- переименование;
- перенос в выбранный каталог;
- удаление файла, пустой папки или подтверждённое рекурсивное удаление дерева.

Имена до 31 байта M8 отображаются и прокручиваются посимвольно; курсор, длина и байтовый индекс совпадают. Имена и строка поиска используют ту же раскладку, что языковые редакторы: цифры без префикса вводят цифры, K + цифровая клавиша включает SMS-ввод, ПП вводит пробел, F + цифровая клавиша - спецсимвол. Cx выполняет забой, а F + Cx очищает поле целиком.

FOCAL, Tiny BASIC и M61 не содержат собственных плоских браузеров. Они вызывают общий API `program_store_choose_file()`, `program_store_choose_directory()` и `program_store_choose_save_target()`. В диалоге можно переходить по дереву и создавать папки; редактор сохраняет файл в выбранный каталог. В меню Разработка -> Файлы МК-61 доступны отдельные операции открытия и сохранения текущей программы. Физические клавиши LOAD/SAVE и числовые аргументы терминальных `load`/`save` сохранены как быстрый интерфейс к корневым файлам `0.m61`…`99.m61`.

Пути и терминал

Единый path API принимает `/` и `\`, абсолютные и относительные пути, повторные разделители, `.` и `..`. Путь с пробелами можно заключить целиком в одинарные или двойные кавычки. Расширение задаёт тип файла; имя без расширения разрешается только при единственном совпадении basename, иначе возвращается ошибка ambiguity.

Интерактивный терминал хранит текущий каталог и показывает его в prompt. Основные команды:

```
pwd
cd [path]
ls [path]                  # aliases: dir, fsls
mkdir [-p] path
mv source destination
rm [-r] path               # aliases: del, fsrm
rmdir path
df                          # alias: fsstat
save path[.m61]
load path[.m61]
open path
run path
```

Например, `open /Pictures/scheme.wbmp` вызывает тот же просмотрщик, что и короткое `OK` в Проводнике. Если выключены и Markdown, и WBMP viewer, файл остаётся видимым, копируемым и переносимым, но команда просмотра возвращает ошибку: отключение viewer не удаляет тип из C9. `open /Games/PONG.ch8` тем же маршрутом передаёт сырой ROM обработчику `C1`. `open /Docs/readme.md` передаёт M8-документ обработчику `T2`: LCD1602 показывает plain text, а UC1609 или USB-экран — форматированную версию. На LCD1602 запуск WBMP и CHIP-8 возможен только при активном USB-экране; без него оболочка сообщает двумя строками: ГРАФИКА / НЕДОСТУПНА. Команда `open /Apps/demo.app` выделяет образ сверху свободной динамической RAM и вызывает его команду `APPLICATION_RUN`.

`mv` не перезаписывает существующий объект. `rm` удаляет файл, `rm -r` -- всё дерево, `rmdir` -- только пустую папку. `df` выводит измеренную ёмкость flash, занятые/свободные узлы, число файлов и каталогов, виртуальный размер кластера и резерв настроек. Все изменяющие дерево команды запрещены внутри M61-сценариев явным allowlist.

Защита записи при просадке питания

На STM32F401/F411 аппаратный PVD закрывает новые записи до того, как VDD достигнет минимальных 2,7 В внешней W25Q128. При низком напряжении USB-диск динамически становится read-only: новый пакет записи или sync получает ошибку, а нижний драйвер повторно проверяет питание перед каждой командой NOR program/erase. После восстановления требуется 100 мс устойчивого питания.

Уже начатая физическая операция не прерывается и не считается гарантированно завершённой. C9 при следующем запуске принимает только последнюю полностью зафиксированную транзакцию; незавершённый batch отбрасывается штатным recovery. Состояние, события и отклонённые операции видны в строке `POWER` команд `df` и `prof`. Подробный контракт описан в `doc/design/STM32-power-monitor.md`.

Динамический FAT12

USB-хост видит стандартный FAT12 superfloppy без partition table: Проекцию FAT, LFN, M8↔UTF-8/UTF-16, проверку и многошаговый commit выполняет одна и та же реализация: на F411 она по умолчанию входит в resident, а на F401 хранится в `/System/USBDISK.APP`. F411 также может использовать внешний APP при `MK61_EXTERNALIZE_USBDISK=1`. Внешний режим resident оставляет USB MSC/SCSI/BOT, отложенный мост в главный цикл и ограниченные атомарные операции C9; на весь MSC-сеанс APP закрепляется в динамической RAM и не может быть вытеснено. В штатном режиме F411 APP-арена для работы USB-диска не нужна.

Серийный номер FAT-тома выводится из стабильного public identity конкретного STM32 в отдельном домене. Поэтому он не меняется после reset или обновления прошивки, но различается у двух калькуляторов с одинаковой W25Q. Старый детерминированный serial по ёмкости остаётся fallback только для host-сборок и неподдерживаемых платформ без UID.

- 512 bytes per sector;
- 1 reserved sector;
- 2 FAT;
- media descriptor `0xF8`;
- фиксированный root directory на 512 entries;
- volume label `MK61S C9`;
- строго не больше 4084 data clusters.

Порог FAT12 определяется числом кластеров, а не экспериментальной CHS-формой дискеты. Геометрии 2M и других floppy-драйверов подтверждают, что BPB может быть нестандартным, но для современных USB Mass Storage C9 сохраняет обычную семантику FAT12 и не зависит от CHS.

Размер FAT-кластера выбирается степенью двойки от 512 байт до 32 КиБ так, чтобы virtual disk не превышал физическую ёмкость и содержал не более 4084 кластеров. На 512 КиБ это 512 байт, на 16 МиБ — 4096 байт. Размер кластера не задаёт физическую потерю на файл: payload упакован в журнал.

ID и FAT-кластер независимы. Непустые файлы и каталоги получают устойчивую F9CH-цепочку; пустой файл — первый кластер 0. Цепочка считается по внешнему размеру, включая рост M8→UTF-8. Продолжения inode не расходуют. Изменение хостом FAT обрабатывается отдельно от committed C9 mapping.

Root directory остаётся фиксированной FAT12-областью на 512 entries (16 КиБ). Четыре заняты системно: volume label и LFN/short-записи маркера `.metadata_never_index`, запрещающего Spotlight индексировать диск. Для пользовательских объектов остаётся 508 entries: от 101 объекта при самом длинном имени (четыре LFN + short) до 254 объектов, чьё видимое имя занимает не более 13 UTF-16 units (одна LFN + short). C9 считает entries точно и атомарно отклоняет create/move/rename, который переполнил бы root; уже созданные объекты при этом не меняются. Полная квота ID доступна через произвольные подкаталоги, которые растут обычными FAT-цепочками. Компактный root принципиален для скорости: macOS может переписывать значительную часть фиксированного root при каждом создании файла.

Подкаталоги содержат стандартные `.` и `..`. Short alias строится из стабильного ID (`Fxxxx` для файла, `Dxxxx` для каталога), поэтому не коллидирует после rename. LFN сохраняет пользовательское имя. Короткая 8.3-запись обязательна в FAT12: именно она хранит атрибуты, первый кластер и размер. LFN дополняет её; это не совместимость со старой внутренней ФС. Импорт принимает и корректные host записи без LFN, в том числе с `NTRes` lower-case flags.

USB staging и sync

На flash 512 КиБ длинный USB-сеанс дополнительно сохраняется из главного цикла: первый промежуточный import запускается при 64 постоянных staged секторах, следующий — после добавления ещё одной полной порции cache. Проверка в USB IRQ читает только RAM; пакет остаётся закреплённым в BOT buffer до foreground обработки. На более крупных flash этот порог отключён. Перед записью снова проверяется PVD, в том числе после длительного import.

Промежуточный import сохраняет данные, которые ОС уже записала, но ещё не связала с FAT и записью каталога. Ошибка проверки незавершённого дерева сохраняет журнал и позволяет хосту дописать метаданные. После успешной публикации удаляются только применённые payload sectors; FAT, каталоги и ещё не привязанные данные остаются видимыми хосту. Окончательное закрытие сеанса освобождает оставшиеся непривязанные сектора.

Хост может записывать FAT, каталог и data sectors в любом порядке. Поэтому принятые 512-байтные блоки сначала объединяются в полностью ассоциативном write-back cache. Его основной размер зависит от геометрии: node и cluster bitmaps, control fields и cache делят один 8-КиБ workspace. Встроенный F411 держит два directory cursor, компактный `USBDISK.APP` — один; cache miss повторяет обход с начала без изменения байтов каталога. На 512 КиБ остаётся 12 основных slots с виртуальным томом 2 МиБ, на 16 МиБ — 7; ещё три временно использует `shared_scratch`. Приостановленный внешний шрифт может дать до 16 дополнительных slots; общий предел — 32. Максимальный DATA OUT chunk выбирается при подготовке MSC как минимум BOT buffer и фактической cache capacity. Поэтому 8-КиБ BOT buffer на F411 может принимать 5-КиБ порции при 10 cache slots; каждая порция принимается атомарно до следующей. APP не вызывается из USB IRQ для запроса этого лимита. Многосекторная WRITE-команда обслуживается такими порциями, поэтому число slots не обязано равняться размеру USB-пакета.

Повторная запись одного LBA заменяет байты в том же cache slot. Если сектор уже dirty, сравнение с постоянными данными откладывается до вытеснения или sync: возврат к исходному содержимому тогда не создаёт запись flash, а ошибка чтения не может затереть ранее принятые байты. До изменения многосекторного BOT-пакета его отложенный обработчик освобождает нужные cache slots; при ошибке вытеснения пакет отвергается целиком. Чтение всегда сначала проверяет cache, поэтому хост видит уже принятые байты. Во внешнем APP-режиме MSC callback возвращает `BUSY`: вызывать APP из USB/IRQ-контекста запрещено. Встроенный F411 может целиком скопировать пакет в свободные RAM slots из callback; если нужно вытеснение или чтение NOR, он тоже возвращает `BUSY`. OUT endpoint остаётся остановленным, а главный цикл передаёт неизменный STM32 BOT buffer обработчику FAT — закреплённому `USBDISK.APP` во внешнем режиме или resident-коду в штатном F411. Отдельной 8-КиБ копии пакета в `.bss` нет.

Во встроенном F411 текстовый экспорт использует проверенный декодированный payload и курсор между M8 и UTF-8 позициями. Последовательные 512-байтовые чтения не запускают повторное чтение или ZX0-декодирование всего файла. Обратный seek пересчитывает позицию по проверенной RAM-копии. Сектор может заканчиваться внутри UTF-8 символа; следующий запрос продолжает его байты. При недоступном кеше остаётся полный CRC-проверяемый потоковый путь.

Импорт F411 преобразует входные данные порциями по 64 байта. Незавершённый символ на границе порции остаётся для следующей порции; конец файла, невалидный UTF-8 и неподдерживаемый Unicode строго проверяются до APPLY. ASCII и обычные русские блоки из четырёх символов преобразуются 32-битными операциями. После неудачного распознавания блока восемь символов проходят скалярно, чтобы не проверять каждый символ чередующегося текста повторно. Для коротких UTF-16 LFN имён быстрый prefix продолжается строгим скалярным suffix после первого неоднородного блока. SIMD32 инструкции не добавлены. F401 и внешний USBDISK сохраняют компактный скалярный путь, вмещающийся в исходные ROM/APP/RAM ограничения.

Имена на томе C9 хранятся в M8, а в FAT LFN показываются как UTF-16LE. Содержимое M61, FOCAL, TinyBASIC, TEXT, STATE и Markdown при экспорте преобразуется из M8 в UTF-8, а при импорте — обратно. APP, FMK, WBMP, CHIP-8 и другие бинарные типы передаются без перекодирования. Невалидный UTF-8 или символ, которого нет в M8, отклоняет весь текстовый файл до изменения C9.

Устройство объявляет `WCE=1` стандартной SCSI Caching mode page `0x08`. Как съёмный носитель (`INQUIRY.RMB=1`) оно также объявляет Removable Block Access Capabilities mode page `0x1B`: один LUN (`TLUN=1`), без system-floppy, FORMAT-progress и механического changer. Ответ `0x3F` возвращает обе страницы по возрастанию кода. Все dirty slots переносятся в persistent staging log при `SYNCHRONIZE CACHE`, при `START STOP UNIT` с остановкой/eject, при выходе из режима USB и при LRU вытеснении заполненного

cache. SCSI sync подтверждается только после завершения переноса и завершения C9 import. В активном сеансе принятые FAT и directory sectors остаются в staging: сохраняются все кластеры каталогов, включая нулевые секторы после end marker. Позиции short entries и aliases не меняются под открытыми vnode ОС. Payload после публикации освобождается; canonical LFN и рост canonical directory chains выполняются при завершении сеанса. Перед import три scratch-slot сначала становятся persistent, `shared_scratch` освобождается для сборки файла, а после commit снова присоединяется к cache. Поэтому unmount/eject ОС и `ESC` на устройстве являются точками сохранения; только аварийное питание до sync может потерять ещё не сброшенный write-back, как у обычного накопителя с WCE.

`START STOP UNIT` с `LOEJ=1, START=0` до ответа выполняет финальный import, canonicalization и очистку журнала, затем помечает SCSI medium извлечённым. После завершения этого запроса устройство закрывает USB-диск и возвращает CDC. macOS Disk Arbitration может логически извлечь том, не передав устройству `LOEJ`; тогда выход выполняется по `ESC`. Один `SYNCHRONIZE CACHE` не закрывает сеанс: та же команда используется для обычного `fsync`.

Повторный sync использует сертификат уже применённого host-view, связанный с ревизией каталога C9. Изменения стандартных timestamps не меняют имён, видов, длин и цепочек; все байты LFN по-прежнему значимы. IRQ сравнивает только принятую RAM, а чтение постоянных CRC-секторов выполняется в foreground. При непроверенных изменениях старая карта служебных файлов не используется.

Проходы RESERVE/RESERVE_NAMES могут доказать, что поддерживаемое дерево не изменилось: каждый ID уникален, имена, родители, виды, типы, длины и полные цепочки совпадают, а видимые данные файлов не содержат staged секторов. Тогда изменение timestamps или игнорируемого служебного каталога не требует нового WAL/import. Настоящие изменения проходят полную проверку и публикацию. `MK61_C9_METADATA_PRECHECK` включён для resident F411 и выключен для внешнего USBDISK/F401. Adaptive batching служебных файлов учитывает оставшиеся live-записи и запас перед новым пакетом, чтобы полный scan не повторялся на каждом структурном пакете.

Persistent staging отделяет порядок FAT-транзакции хоста от атомарных операций C9. Один staging-сектор содержит 16-байтный header и семь записей вида:

```
16-byte record header + 512-byte payload
```

Header хранит LBA-key, поколение, state и CRC32. Header и 512-байтный payload программируются одной проверяемой stream-операцией; при recovery запись считается `ACTIVE` только после проверки CRC payload. Поэтому оборванный page-program не вытесняет предыдущее поколение и не требует отдельной commit-записи в NOR. На обычной flash до 441 последнего live-блока индексируются в RAM одним сортированным packed-массивом `LBA + record-ref`. Lookup выполняется двоичным поиском; вставка/удаление сдвигают только индекс; 65-й сектор служит резервом атомарной компактизации. На 512-КиБ варианте 16 постоянных рабочих секторов содержат 112 записей, а 17-й является резервом. При заполнении постоянной области журнал заимствует свободные сектора C9-данных: индекс вмещает до 640 разных блоков (320 КиБ полезной нагрузки). Выделение никогда не трогает живые файлы, текущий сектор записи и резерв сборки мусора; ещё минимум 12 свободных erase-секторов остаются для публикации файлов при `SYNC`, дополнительно к current writer и GC reserve. При фрагментации штатный COW GC сначала сливает живые записи: выбор пустого victim не освободил бы дополнительный физический сектор. Затем список защищённых секторов вычисляется заново. Это не обещание принять произвольные 320 КиБ на заполненном томе: предел зависит от реально свободной flash.

Заимствованный сектор помечен заголовком `C9S0` с эпохой тома. Аллокатор C9 и сборщик мусора не стирают такие сектора, в том числе после перезагрузки до восстановления RAM-индекса журнала. После финального commit или явной отмены заимствованные сектора стираются и возвращаются C9. Обрыв питания во время расширения, уплотнения или возврата секторов не публикует неполную запись и не стирает старую действительную версию блока.

Перед append C9 сравнивает новый сектор с эффективным содержимым всего тома, включая data area. Повторная запись уже совпадающих FAT/root/data-блоков подтверждается без program/erase NOR. Это особенно сокращает macOS zero-fill и повторные записи AppleDouble, одновременно уменьшая износ flash.

Finder создаёт 4-КиБ AppleDouble-sidecar `._имя` для extended attributes. Такой временный файл может иметь то же расширение `.m61`, `.foc` и т. п., но не является программой. Импорт распознаёт префикс `._`, не включает sidecar в C9 и не позволяет ему сорвать commit настоящего файла. Неизвестные расширения также игнорируются до проверки C9-квоты размера. На всех поддерживаемых размерах тома накопитель распознаёт `._*` и при sync либо заполнении половины постоянного staging обновляет карту ролей,

освобождая sidecar payload. Последующие секторы подтверждённой sidecar-цепочки игнорируются. Обычные USB metadata writes группируются до этой проверки, чтобы изменения timestamp не запускали полный проход каталога при каждом пакете. После изменения FAT или dirent старая ignore-карта временно не применяется: бывший sidecar-кластер может уже принадлежать обычному файлу. Пакеты принимаются в cache целиком с учётом этой возможной смены роли. Прямая синхронная `write_sector` обновляет роли сразу; USB write-back делает это до подтверждения durable sync.

Полные FAT12-карты sidecar, обычных файлов и каталогов существуют только на стеке во время ограниченного прохода каталога. Между проходами роли кластеров хранятся в общей двухбитовой рабочей карте, которую commit использует для проверки владения кластерами. Отдельная двухбитовая карта описывает node IDs. Это сохраняет раннюю фильтрацию и на штатном 16-МиБ томе, не увеличивая постоянную RAM USB-обработчика. После успешной или отклонённой синхронизации карта очищается; если MSC-сеанс продолжается, она восстанавливается из актуальных FAT и каталогов на следующей границе sync или staging pressure.

При `SYNC` virtual FAT выполняет проверку и многошаговый APPLY:

1. Проверяет FAT-цепочки, LFN, уникальность cluster-ID, глубину, размеры и наличие всех data-секторов и directory tree без публикации payload. Для каждого APP read-only preflight проверяет заголовок ABI 6, размеры контейнера и CRC хранимого потока. Во внешнем режиме `USBDISK.APP` занимает исполняемую арену и не может там же развернуть второй APP; штатный F411 арену не занимает, но использует тот же быстрый preflight. Перед первым исполнением обычный загрузчик всё равно полностью декодирует ZX0/BCJ, применяет релокации и проверяет CRC готового образа. Неполный файл или невалидный APP отклоняет текущий sync до изменения любого узла C9. Пока MSC-сеанс открыт, staging сохраняется: хост ещё может дописать, удалить или заменить отклонённый файл и повторить sync.
2. Создаёт/перемещает каталоги, собирает staged data и атомарно публикует файлы через C9 WAL.

При удалении файл unlink-ится одной WAL-записью. Его data/mapping возвращает GC; отдельного inode для продолжения и поиска осиротевших extents нет. Directory chains обновляются неизменяемой F9CH-записью, включая освобождение ненужного хвоста. Сохранённый staging позволяет повторить незавершённый APPLY.

Повторный sync уже опубликованного host view не запускает APPLY снова, если после него не приняты новые секторные записи и не изменился media revision. При закрытии достаточно завершить canonical directory chains и удалить снимки; reverse/name projection для следующего сеанса строится при его открытии.

После полного APPLY и до canonicalization в staging записывается служебный `C9CLOSE` marker с ключом `0x7FFFFF`, за пределами доступных хосту LBA. Marker имеет обычные поколение, CRC и C9 format epoch. Он удаляется последним, после всех FAT/directory snapshots. При reboot с marker восстановление продолжает canonicalization и очистку, сохраняя уже опубликованное дерево C9; частично удалённые snapshots не разбираются как новая FAT-транзакция.

Для многокластерного APP, MK61_BINARY или CHIP-8 ROM APPLY не собирает файл целиком в RAM. `FileSource` по запросу читает нужные 512-байтовые секторы из staging, cache или прежней C9-версии и сразу передаёт их в `C9B0`. Поэтому порядок записи FAT не влияет на буферизацию, а частичное обновление существующей цепочки может прислать только изменившиеся секторы. Терминальный `fsput` использует тот же потоковый путь, сохраняя вход по 512 байт в отдельные staging-key.

Циклы FAT, выход за диапазон, повторное использование cluster-ID, неверный LFN, превышение квоты своего типа (1536 байт, 3584 для TinyBASIC, 1600 для WBMP, 1536/8192 для FMK, 3584 для CHIP-8, 4096 для BIN либо 20 544 для APP) и неподдерживаемая глубина отклоняются. При финальном выходе из USB по `ESC` такая детерминированно невалидная host-транзакция автоматически отменяется: staging очищается, а последний committed C9 остаётся неизменным. Ошибка возвращается и при финализации, поэтому экран не подтверждает потерянный файл как успешно сохранённый, но следующий USB-сеанс уже не блокируется старым batch. После принятого `ESC` или извлечения диска экран показывает сохранение..., пока завершаются проверка и commit; клавишу не требуется держать или нажимать повторно. Retryable ошибки чтения, записи flash или временно недоступного ресурса staging не очищают; после reset он восстанавливается по поколениям и CRC и может быть повторно синхронизирован. Пакет из нескольких файлов не является одной общей транзакцией: после аварийного питания уже завершённые файлы могут быть новыми, остальные — старыми или отсутствовать. При этом каталог остаётся структурно валидным, а повторная передача и `SYNC` сходятся к полному набору. Host-тест проверяет одновременный импорт 25 APP

и моделирует обрывы питания с reboot и повтором.

Настройки

Последний физический erase-сектор всегда зарезервирован под настройки:

```
offset 0..4079       settings journal
offset 4080..4095    C9SG capacity guard
```

Обычная очистка файловой системы и восстановление каталога не стирают журнал настроек. API erase-settings стирает сектор целиком и немедленно восстанавливает guard. При смене измеренной ёмкости сектор переезжает, поэтому создаётся новый пустой журнал.

Гарантии и границы

- Поддерживается только C9; старые readers, migrations и перенос настроек удалены.
- Валидная C9 загружается без тестовых erase/write.
- JEDEC/SFDP -- подсказка; окончательная граница неразмеченной flash подтверждается записью и alias-aware verify.
- Полный файл и отдельная операция каталога публикуются атомарно через WAL.
- Число APP не имеет отдельного compile-time лимита; оно ограничено общей квотой inode, местом журнала данных и ёмкостью каталога FAT.
- FAT12 является представлением объектов, а не побайтовой копией SPI NOR.
- Свободные байты FAT не равны числу свободных физических байтов журнала; отказ определяется отдельными ресурсами ID, FAT, root, data и staging.
- CRC32 обнаруживает случайные повреждения, но не является криптографической защитой.
- При валидном locator и повреждении каталога C9 не пишет во flash и не форматирует её автоматически: `df` предлагает явную команду `format` или Система → Формат диска (с сохранением настроек), либо Система → Полный сброс (с удалением настроек). Во внешнем режиме USB-диска до восстановления `/System/USBDISK.APP` не запускается; это всегда верно для F401 и верно для F411 при `MK61_EXTERNALIZE_USBDISK=1`. В штатном F411 он доступен сразу после форматирования.
- При физической неисправности обеих C9 locator-копий содержимое считается неразмеченным и форматируется; эвристического восстановления нет.

USB import и решение по UNMAP

До изменения каталога выполняются проходы проверки цепочек, резервирования прежних ID и ancestors, имён и ресурсов. Во flash записывается bounded план назначения ID, включая пустые файлы. Коллизии имён проверяются до мутаций по parent и полному имени без учёта регистра. Циклы rename разрываются временными именами. Parser использует итеративный стек до 32 уровней.

До APPLY сохраняются canonical FAT и только изменяемые родительские каталоги. Удалённые объекты освобождают ID/mapping, но старые данные остаются доступны через staged/snapshot источники. Затем публикуются отдельные изменения, при активном сеансе сохраняются точные FAT/dirent snapshots и освобождается payload staging. Финализация канонизирует LFN и directory chains и очищает оставшийся staging. После публикации строится новый reverse index. Невалидный пакет отвергается до APPLY; I/O ошибка сохраняет durable staging для повтора. Весь многофайловый пакет не является одной WAL-транзакцией.

SCSI UNMAP не реализуется и не объявляется. SPI NOR не имеет ATA TRIM; удаление уже известно по FAT import и возвращает место через обычный GC. Прямое стирание LBA невозможно: erase-сектор содержит данные нескольких объектов, metadata и записи восстановления. Возвращаться к UNMAP имеет смысл только при измеренном выигрыше от раннего discard staged данных или при переходе к секторному FTL. Корректность удаления от UNMAP не зависит.